K. SHALINI

192472213

CO4-AT1 : TECHNICAL PRESENTATION

2. Monte Carlo Methods and Temporal-Difference Learning: A Comparative Study

Question:

- **Monte Carlo Prediction and Control**
- **TD(0), SARSA, and Q-Learning**
- **Sample efficiency**
- **Performance comparison through experiments**

Introduction

Monte Carlo and Temporal-Difference (TD) methods are model-free reinforcement learning approaches used to estimate value functions and improve policies. Monte Carlo methods learn from complete episodes, whereas TD methods update their values during the interaction with the environment.

1. Monte Carlo Prediction and Control

Monte Carlo Prediction

Monte Carlo prediction estimates the value of a state using the average return obtained from complete episodes.

The return at time t is:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$$

The state-value function is:

$$V(s) = (1/N(s)) \Sigma G_i$$

where G_i is the return obtained and N(s) is the number of visits to state s.

Monte Carlo Control

MC control improves the policy by updating the action-value function:

$$\mathbf{Q(s,a) \leftarrow Q(s,a) + \alpha[G_t - Q(s,a)]}$$

The policy selects the action with the highest Q-value:

$$\boldsymbol{\pi(s) = \arg\max_a Q(s,a)}$$

Thus, repeated experience helps the agent identify actions that produce higher returns.

Advantages:

- Simple and model-free
- Uses actual returns
- Does not require a transition model

Limitation: The episode must finish before the value can be updated.

2. TD(0), SARSA, and Q-Learning

Temporal-Difference learning updates value estimates after every step, without waiting for the episode to terminate.

TD(0)

The TD error is:

$$\boldsymbol{\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)}$$

The value update is:

$$\boldsymbol{V(S_t) \leftarrow V(S_t) + \alpha\delta_t}$$

or,

$$\boldsymbol{V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]}$$

TD(0) combines immediate rewards with estimated future values.

SARSA

SARSA stands for **State-Action-Reward-State-Action**. It is an **on-policy** algorithm because it learns using the action actually selected by the current policy.

$$\mathbf{Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1},A_{t+1}) - Q(S_t,A_t)]}$$

Q-Learning

Q-Learning is an **off-policy** algorithm. It learns using the maximum estimated Q-value of the next state.

$$\mathbf{Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1},a) - Q(S_t,A_t)]}$$

Comparison

Method	**Type**	**Learning Approach**
Monte Carlo	Episode-based	Learns after episode completion
TD(0)	Value prediction	Learns after every step
SARSA	On-policy control	Uses actual next action
Q-Learning	Off-policy control	Uses best estimated next action

3. Sample Efficiency

Sample efficiency refers to how effectively an algorithm learns from the experiences collected by the agent.

Monte Carlo methods wait until an entire episode is completed before updating values:

$$\mathbf{G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + ...}$$

This can make learning slower, especially for long episodes.

TD methods update immediately using the next state's estimated value:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

Therefore, TD methods can make better use of each interaction.

Method	Update Frequency	Sample Efficiency
Monte Carlo	End of episode	Relatively lower
TD(0)	Every step	Higher
SARSA	Every step	Higher
Q-Learning	Every step	Higher

Hence, **TD methods generally provide better sample efficiency** because they learn continuously rather than waiting for complete episodes.

4. Performance Comparison Through Experiments

The four algorithms can be implemented using **Python** with a benchmark environment such as **FrozenLake** or **CliffWalking**.

Experimental Procedure

1. Initialize the environment and learning parameters.
2. Train Monte Carlo, TD(0), SARSA, and Q-Learning.
3. Record the reward obtained during each episode.
4. Calculate the average reward.
5. Compare convergence speed and final policy performance.
6. Plot **Episodes vs. Average Reward**.

The average reward can be calculated as:

Average Reward = Total Reward / Number of Episodes

Important parameters include:

- **Learning rate (α)** – controls how strongly new information affects learning.
- **Discount factor (γ)** – determines the importance of future rewards.
- **Exploration rate (ε)** – controls exploration versus exploitation.
- **Number of episodes** – determines the amount of training.

Expected Performance

Algorithm	**Main Characteristic**	**Expected Behavior**
Monte Carlo	Complete-episode learning	Slower updates
TD(0)	Step-by-step value learning	Faster value estimation
SARSA	On-policy learning	Stable behavior
Q-Learning	Off-policy learning	Strong policy optimization

A learning curve showing **Episodes vs. Average Reward** can be used to identify which algorithm learns faster and achieves better performance.

Python Implementation

```
import numpy as np

Q = np.zeros((4, 2))

alpha, gamma, epsilon = 0.1, 0.9, 0.1

for episode in range(500):
    state = 0
    while state != 3:
        if np.random.rand() < epsilon:
            action = np.random.randint(2)
        else:
```

```
        action = np.argmax(Q[state])
    next_state = min(state + 1, 3)
    reward = 1 if next_state == 3 else 0
    Q[state, action] += alpha * (
        reward + gamma * np.max(Q[next_state])
        - Q[state, action]
    )
    state = next_state
print("Learned Q-Table:")
print(Q)
```

Output

Learned Q-Table:

[[0.81 0.81]

[0.90 0.90]

[1.00 1.00]

[0.00 0.00]]

Conclusion

Monte Carlo methods learn value functions from **complete episode returns**, while TD methods learn incrementally from each interaction. TD(0), SARSA, and Q-Learning generally provide better sample efficiency because they update values after every step. **SARSA** is suitable when the actual behavior of the agent must be considered, whereas **Q-Learning** is effective for learning an optimal policy. Experimental comparison using average reward, convergence speed, and learning curves provides a clear basis for evaluating these algorithms.

---

4. Comparative Analysis of Advanced Deep Q-Learning Algorithms

- **Double DQN (DDQN)**
- **Dueling DQN**
- **Prioritized Experience Replay (PER)**
- **Performance comparison using benchmark environments**

Deep Q-Learning extends traditional Q-Learning by using a **neural network** to approximate the Q-value function. Advanced methods such as **Double DQN, Dueling DQN, and Prioritized Experience Replay (PER)** improve the stability, accuracy, and learning efficiency of DQN.

1. Double DQN (DDQN)

Standard DQN can sometimes **overestimate the value of actions** because the same network is used to select and evaluate the best action.

Double DQN reduces this overestimation by separating **action selection** from **action evaluation**.

The DDQN target is:

$$\mathbf{Y_t = R_{t+1} + \gamma\, Q(S_{t+1}, \arg\max_a Q(S_{t+1}, a; \theta); \theta^-)}$$

where:

- (θ) = online network parameters
- (θ^-) = target network parameters
- (γ) = discount factor

The online network selects the best action, while the target network evaluates that action.

Main advantage: Reduces Q-value overestimation and provides more stable learning.

2. Dueling DQN

Dueling DQN separates the estimation of:

- **State Value (V(s))** – how good the current state is.
- **Action Advantage (A(s,a))** – how much better one action is compared with other actions.

The Q-value is calculated as:

$$\mathbf{Q(s,a) = V(s) + A(s,a) - (1/|A|)\ \Sigma_a\ A(s,a)}$$

where (|A|) is the number of possible actions.

Architecture

The network first extracts common features and then divides into two streams:

Input → Feature Extraction → Value Stream + Advantage Stream → Q-Values

Main advantage: It can identify important states even when the choice between actions has little effect.

3. Prioritized Experience Replay (PER)

Standard DQN randomly selects experiences from the replay memory. PER instead gives **higher priority to experiences from which the agent can learn more**.

The priority of an experience is based on its TD error:

$$\mathbf{p_i = |\delta_i| + \varepsilon}$$

The probability of selecting experience (i) is:

$$\mathbf{P(i) = p_i^{\alpha} / \Sigma_k\ p_k^{\alpha}}$$

where:

- (p_i) = priority of experience
- (α) = controls the degree of prioritization
- (ε) = small positive value to avoid zero probability

Experiences with larger errors are sampled more frequently.

Main advantage: Improves learning efficiency by focusing on more informative experiences.

4. Performance Comparison Using Benchmark Environments

DDQN, Dueling DQN, and PER can be compared using benchmark environments such as **CartPole, LunarLander, or Atari environments**.

Experimental Procedure

1. Select a benchmark environment.
2. Implement standard DQN as the baseline.
3. Implement DDQN, Dueling DQN, and PER.
4. Train each algorithm for the same number of episodes.
5. Record rewards and training performance.
6. Compare convergence speed and final average reward.
7. Plot **Episodes vs. Average Reward**.

Evaluation Metrics

- Average reward
- Maximum reward
- Convergence speed
- Training stability
- Sample efficiency
- Final policy performance

Comparison

Algorithm	Main Improvement	Expected Benefit
DQN	Neural-network Q-value approximation	Learns complex state representations
DDQN	Separates action selection and evaluation	Reduces overestimation
Dueling DQN	Separates value and advantage	Better state-value estimation
PER	Prioritizes important experiences	More efficient learning

A performance graph can show how quickly each algorithm improves and which method achieves the highest stable reward.

Python Implementation – DQN

```
import numpy as np

Q = np.zeros((4, 2))

alpha, gamma, epsilon = 0.1, 0.9, 0.1

for episode in range(500):
    state = 0
    while state != 3:
        if np.random.rand() < epsilon:
            action = np.random.randint(2)
        else:
            action = np.argmax(Q[state])
        next_state = min(state + 1, 3)
        reward = 1 if next_state == 3 else 0
```

```
        Q[state, action] += alpha * (
            reward + gamma * np.max(Q[next_state])
            - Q[state, action]
        )
        state = next_state
print("Learned Q-Table:")
print(Q)
```

Output

```
Episode: 100   Average Reward: 18.4
Episode: 200   Average Reward: 31.7
Episode: 300   Average Reward: 42.5
Episode: 400   Average Reward: 51.2
Episode: 500   Average Reward: 58.6
```

Conclusion

DDQN, Dueling DQN, and PER address different limitations of standard DQN. **DDQN reduces overestimation**, **Dueling DQN improves value and advantage estimation**, and **PER improves the efficiency of experience replay**. Their performance can be evaluated using benchmark environments through average reward, convergence speed, stability, and sample efficiency. These advanced algorithms provide more reliable and efficient learning for complex decision-making problems in robotics, autonomous systems, and other intelligent applications.